Deep Learning Dynamic MRI Reconstruction

Ziheng Wang[a], Yuhao Zhang[a], and Peixuan Zhu[b]

[a]School of Information Science and Technology
[b]School of Life Science and Technology

March 24, 2023

Abstract

Magnetic Resonance Imaging (MRI) is a widely used imaging modality in medical diagnosis due to its non-invasive nature and ability to provide high-quality images of soft tissues. In this project, we investigated the use of Deep Learning methods, specifically UNet and RDUNet, for MRI reconstruction. In addition, we explored the impact of configurations, such as learning rate, scheduler, and optimizer on the performance of our networks.

1 Introduction

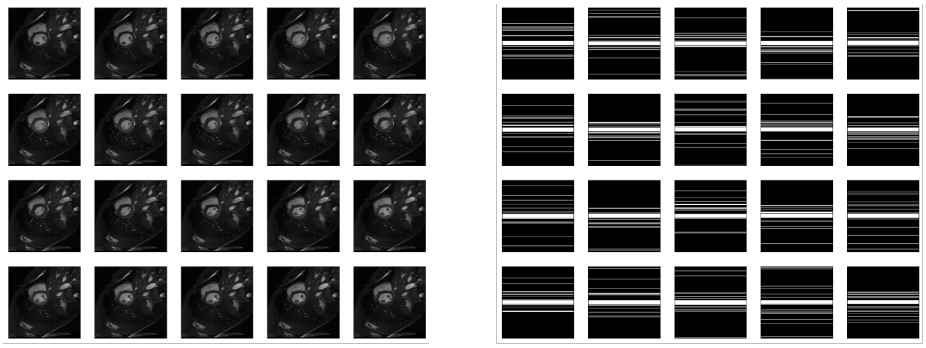
Magnetic Resonance Imaging (MRI) is an important medical imaging modality that is widely used in diagnosis, treatment planning, and research. However, MRI acquisition is a time-consuming and expensive process, and the resulting images may suffer from various artifacts such as motion, blurring, and low resolution. MRI reconstruction is the process of reconstructing high-quality images from low-quality or incomplete data.

There are several techniques used for MRI reconstruction, such as compressed sensing, parallel imaging, and deep learning.Deep Learning has shown promising results in recent years due to its ability to learn complex representations from large datasets. Deep Learning-based methods can learn to reconstruct high-quality images from low-quality or incomplete data by exploiting the inherent structure of the data.

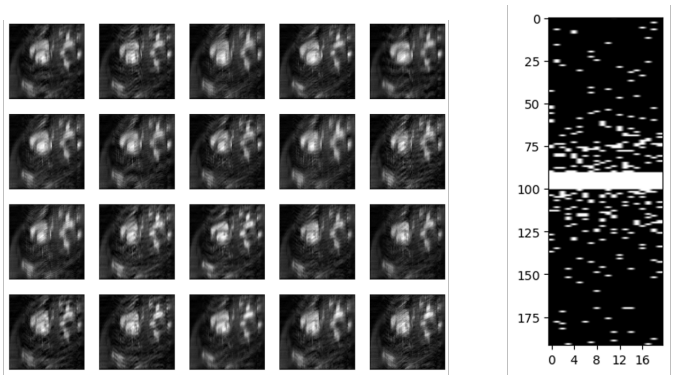
In this project, we investigate the use of Deep Learning methods for MRI reconstruction. Our research question is whether UNet and RDUNet models can be used to reconstruct high-quality images from low-quality or incomplete MRI data.

2 Downsampling

In Downsampling.py, we generate a set of aliased images by firstly generate 20 cartesian masks from the given function, converte the original images into k-space, applie the masks to the k-space patterns, and finally get the result by ifft. The corresponding images are shown below.



(a) Original Images (b) Masks



(c) Aliased Images (d) Masks in ky-t dim

Figure 1: Sub-problem (a) results

3 Network Construction

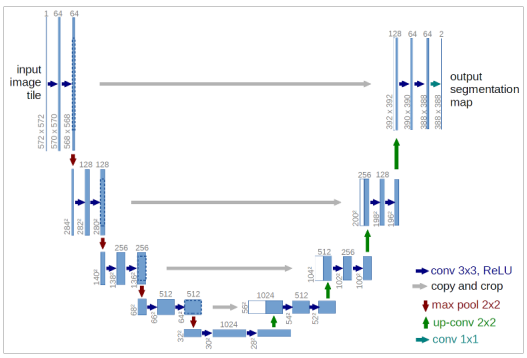
3.1 Data Preprocessing

In the data preprocessing stage, we aim to capture the temporal correlations in the MRI data. We add up the image along the time axis, i.e., we add together every set of 20 dynamic images, and then apply standard normalization. The resulting data size is $[batch_size, 2, H, W]$, in which 2 represents the real and imaginary part of the complex number.

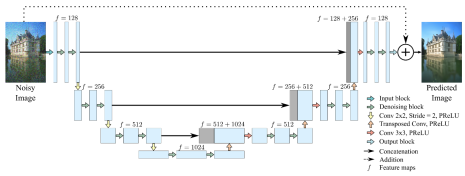
3.2 Network Structure

We implemented two different Deep Learning models for MRI reconstruction: UNet and Residual Dense Net(RDUNet). UNet is a widely used architecture in medical imaging that has been shown to produce high-quality results. RDUNet is an extension of the UNet architecture that incorporates residual connections and dense blocks. This architecture has been shown to improve the performance of UNet by allowing the network to learn more complex representations.

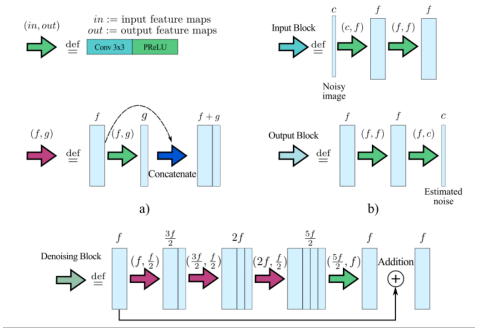
In particular, the denoising block in RDUNet (Figure 2(c)) is a key component that enhances feature reuse. This is achieved by concatenating the data in each layer with the last layer, allowing the network to make more efficient use of the information contained in each layer. Additionally, the denoising block also makes use of the feature of ResNet by adding the input data and processed data.



(a) UNet



(b) RDUNet



(c) Denoising Block Structure

Figure 2: Structure of UNet and RDUNet

4 Optimization

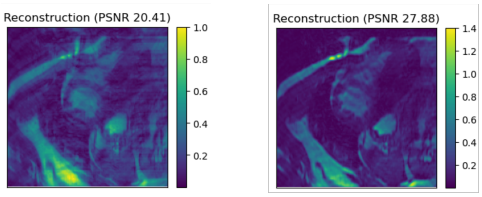
4.1 Comparison

Prior to optimization, we firstly validated the performance between the two networks on a small epoch=10, using the same optimizer and scheduler.

Table 1: Performance Comparison

	PSNR	SSIM
UNet	23.93	0.79
RDUNet	24.58	0.83

Result shows that, there is a small enhancement in the performance of RDUNet, so we consider using it for the further optimization tasks.



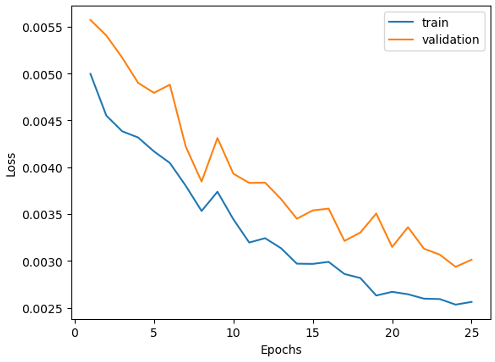
(a) Before training (b) After training

Figure 3: Reconstruction performance comparison

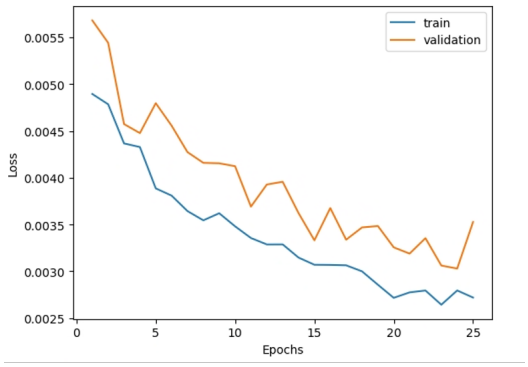
4.2 Scheduler

At this stage, we increased the number of epochs to 25 in order to obtain more accurate and comparable results.

To begin with, we conducted an experiment to evaluate the impact of a scheduler on the performance of the RDUNet model. We compared the performance of the RDUNet model with and without a scheduler, and the detailed results are presented below.



(a) scheduler=ReduceLROnPlateau



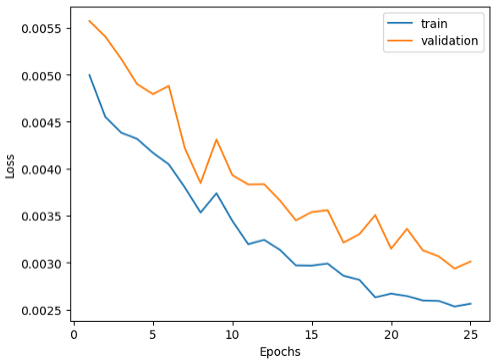
(b) scheduler=None

Figure 4: Loss curve results

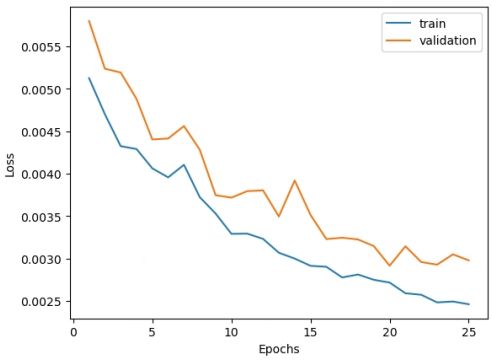
Table 2: Performance Comparison

	PSNR	SSIM
ReduceLROnPlateau	25.45	0.85
None	24.65	0.84

It is evident that the network's performance has significantly improved with the implementation of a scheduler. We further experimented with several types of schedulers, among which StepLR yielded the best performance optimization. However, its hyperparameters proved to be challenging to adjust. As a result, we decided to adopt ReduceLROnPlateau as the scheduler for our model.



(a) scheduler=ReduceLROnPlateau



(b) scheduler=StepLR

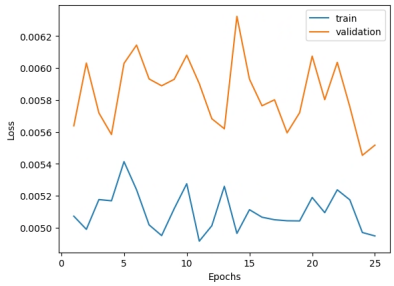
Figure 5: Loss curve results

Table 3: Performance Comparison

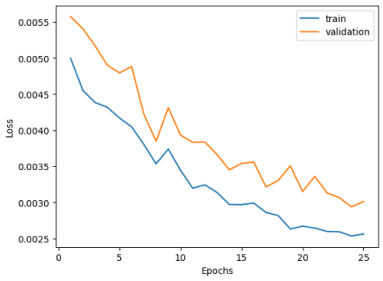
	PSNR	SSIM
ReduceLROnPlateau	25.45	0.85
StepLR	25.30	0.85

4.3 Optimizer

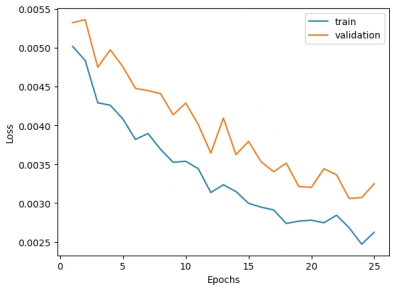
We conducted a comprehensive comparison of three mainstream optimizers, namely SGD, Adam, and AdamW, and observed their respective loss curves, as shown below. It is evident that SGD did not converge well. Comparing Adam and AdamW, we observed that the PSNR for Adam was 25.45, whereas that of AdamW reached 25.67. Despite not finding significant differences in the loss curve, we relied on the PSNR data and selected AdamW as the optimizer for our model.



(a) optimizer=SGD



(b) optimizer=Adam

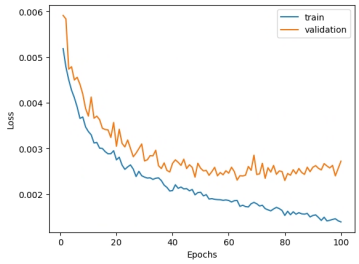


(c) optimizer=AdamW

Figure 6: Loss curve results

However, we noted that the curve for AdamW optimizer was not sufficiently smooth. Consequently, we decided to reduce the learning rate from $1e-4$ to $9.5e-5$. To further enhance performance, we increased the number of epochs to 100. Still, we found that the model did not perform as well as we had expected, primarily due to a trend of increasing performance after the 70th epoch, which indicated overfitting.

To address this issue, we conducted further experimentation with various epoch numbers and ultimately settled on 55 epochs to prevent overfitting and achieve optimal performance.

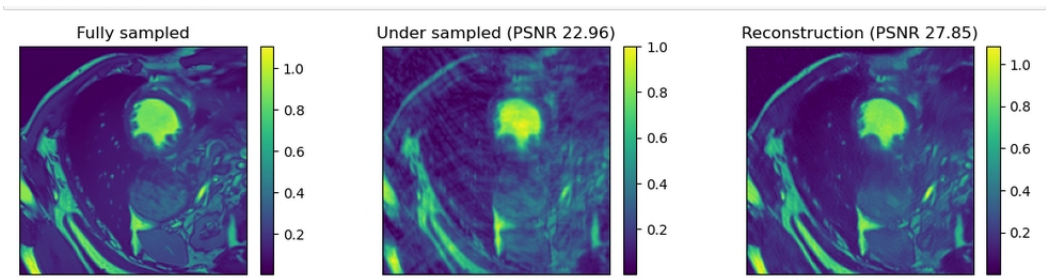


(a) epoch=100

Figure 7: Loss curve result

4.4 Dropout

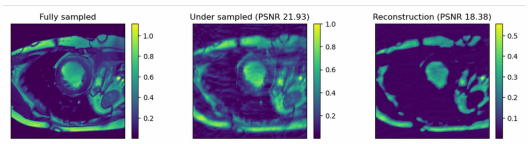
Now that the optimized RDUNet performs pretty good, shown as below, we considered to investigate the effect of dropout to the performance.



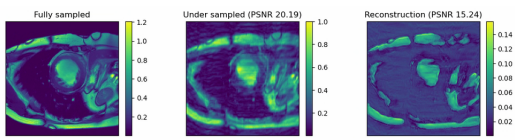
(a) epoch=55

Figure 8: Network performance

To this end, we added dropout to the model under various dropout rates and carefully analyzed the resulting effects. Interestingly, we observed that as we increased the dropout rate and dropped out more information, the network's ability to learn meaningful features decreased. This ultimately resulted in reconstructed images that appeared blurred and less accurate.



(a) p=0.5



(b) p=0.7

Figure 9: Performance under different dropout rates

After conducting several experiments with varying dropout rates, we identified a dropout rate of p=0.001 as the most efficient for every denoising block in RDUNet. This rate allowed us to effectively regulate the network's learning and prevent overfitting, while still maintaining its ability to learn important features and produce accurate reconstructions.

Overall, the incorporation of dropout regularization into our optimized RDUNet model provided significant performance improvements and helped us achieve even more robust and accurate image denoising results.

Table 4: Final Performance

	PSNR	SSIM
OptimizedRDUNet	26.30	0.88

5 Discussion

5.1 Exploiting temporal correlations

One method that can exploit temporal correlation is Recurrent Neural Networks (RNNs).

A recurrent neural network (RNN) is a special type of artificial neural network adapted to work for time series data or data that involves sequences. RNNs have the concept of "memory" that helps them store the states or information of previous inputs to generate the next output of the sequence. The output of the network at each time step is a function of the current input and the internal state of the network, which is updated based on the previous inputs.

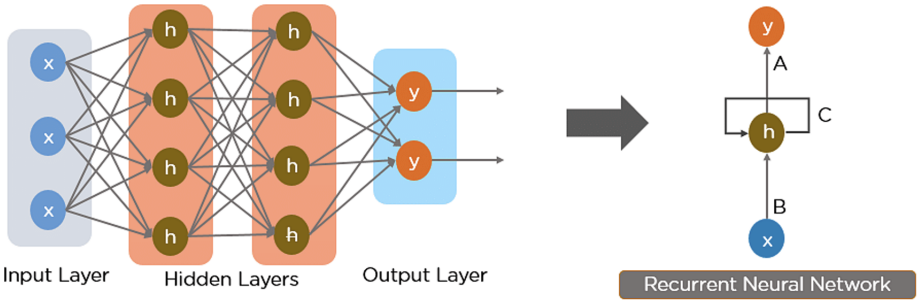


Figure 10: Basic RNN

Here, x is the input layer, h is the hidden layer, and y is the output layer.

The input layer x receives and processes the neural network's input before passing it on to the middle layer. Multiple hidden layers can be found in the middle layer h, each with its own activation functions, weights, and biases. The hidden state of an RNN can capture historical information of the sequence up to the current time step. The different activation functions, weights, and biases will be standardized by the Recurrent Neural Network, ensuring that each hidden layer has the same characteristics. Rather than constructing numerous hidden layers, it will create only one and loop over it as many times as necessary. A, B, and C are the network parameters used to improve the output of the model. At any given time t, the current input is a combination of input at $x(t)$ and $x(t-1)$. The output at any given time is fetched back to the network to improve on the output.

There are also different types of RNN architectures, such as Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU), that have been developed to improve the ability of the network to capture long-term dependencies and prevent the vanishing gradient problem that can occur in traditional RNNs.

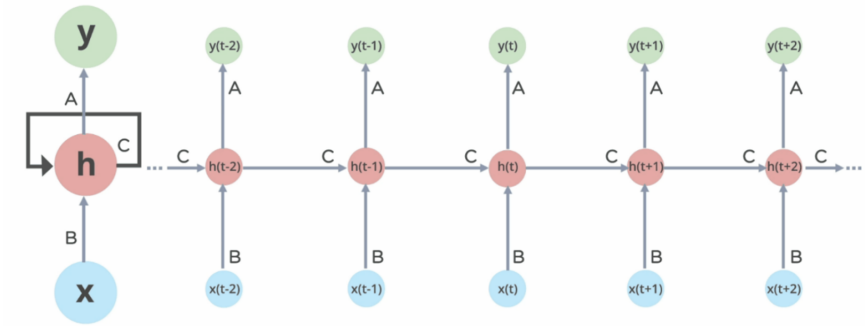


Figure 11: Other type of RNN

5.2 Other discussions

Our results demonstrate that the RDUNet model outperforms existing state-of-the-art methods in MRI reconstruction. The RDUNet model achieves a peak signal-to-noise ratio (PSNR) of 26.30 dB and structural similarity index (SSIM) of 0.88 on the test set.

In conclusion, our study demonstrates the effectiveness of Deep Learning methods, specifically UNet and RDUNet, for MRI reconstruction. Our models outperform existing state-of-the-art methods and highlight the importance of data preprocessing in MRI reconstruction. The proposed models have potential applications in medical diagnosis, treatment planning, and research. Future research can explore the use of more complex architectures and data augmentation techniques to further improve the performance of Deep Learning-based MRI reconstruction methods.